

LAB-8.3

2303A51123

BATCH-10

Task 1: Email Validation using TDD

Scenario: You are developing a user registration system that requires reliable email input validation.

Prompt used:

generate a python function for developing a user registration thar requires user input validation

it must contains @ and . character and must not start or end with any special charcter

it should not allow multiple @

the function should return true if the email is valid and false if it is invalid.

```
lab-8.3.py X
lab-8.3.py > ...
1 #generate a python function for developing a user registration thar requires user input validation
2 # it must contains @ and . character and must not start or end with any special charcter
3 # it should not allow multiple @
4 # the function should return true if the email is valid and false if it is invalid
5 def validate_email(email):
6     # Check if email contains @ and . characters
7     if '@' not in email or '.' not in email:
8         return False
9
10    # Check if email starts or ends with a special character
11    if email[0] in ['@', '.'] or email[-1] in ['@', '.']:
12        return False
13
14    # Check for multiple @ characters
15    if email.count('@') > 1:
16        return False
17
18    return True
19 # Example usage
20 email = input("Enter your email address: ")
21 if validate_email(email):
22     print("Email is valid.")
23 else:
24     print("Email is invalid.")
25
26
```

```
PROBLEMS 100% Console DEBUG CONSOLE TERMINAL FORMS OUTPUTS SPELLCHECKER Python
PS C:\Users\hp\OneDrive\Desktop\AI> & C:/Users/hp/AppData/Local/Microsoft/WindowsApps/python3.11.exe c:/Users/hp/OneDrive/Desktop/AI/lab-8.3.py
Enter your email address: vamshipuvati@gamil.com
Email is valid.
PS C:\Users\hp\OneDrive\Desktop\AI>
```

Task 2: Grade Assignment using Loops

Scenario: You are building an automated grading system for an online examination platform.

Prompt used:

generate a python function for automated grading system

if the grade is 90-100 print A, 80-89 print B, 70-79 print C, 60-69 print D, below 60 print F

include boundary values 60, 70, 80, 90 in the test cases

```
# generate a python function for automated grading system
# if the grade is 90-100 print A, 80-89 print B, 70-79 print C, 60-69 print D, below 60 print F
# include boundary values 60, 70, 80, 90 in the test cases
def calculate_grade(score):
    if 90 <= score <= 100:
        return 'A'
    elif 80 <= score < 90:
        return 'B'
    elif 70 <= score < 80:
        return 'C'
    elif 60 <= score < 70:
        return 'D'
    elif score < 60:
        return 'F'
    else:
        return 'Invalid score'

score = int(input("Enter the score: "))
grade = calculate_grade(score)
print(f"The grade is: {grade}")
```

```
47
PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS SPELL CHECKER 1 Python + -
PS C:\Users\hp\OneDrive\Desktop\AI> & C:/Users/hp/AppData/Local/Microsoft/windowsApps/python3.11.exe c:/Users/hp/OneDrive/Desktop/AI/lab-8.3.py
Score: 59, Grade: F
Score: 60, Grade: D
Score: 69, Grade: D
Score: 70, Grade: C
Score: 79, Grade: C
Score: 80, Grade: B
Score: 89, Grade: B
Score: 90, Grade: A
Score: 100, Grade: A
PS C:\Users\hp\OneDrive\Desktop\AI>
```

Task 3: Sentence Palindrome Checker

Scenario: You are developing a text-processing utility to analyze sentences.

Prompt used:

generate a python program for if the sentence is a palindrome or not

ignore spaces and punctuation

```
lab-8.3.py X
lab-8.3.py > ...
48 C:\Users\hp\OneDrive\Desktop\AI\lab-8.3.py if the sentence is a palindrome or not
49 # ignore spaces and punctuation
50 #don't give comments in the code.
51 def is_palindrome(sentence):
52     # Remove spaces and punctuation
53     cleaned_sentence = ''.join(char for char in sentence if char.isalnum()).lower()
54
55     # Check if the cleaned sentence is equal to its reverse
56     return cleaned_sentence == cleaned_sentence[::-1]
57 # Example usage
58 sentence = input("Enter a sentence: ")
59 if is_palindrome(sentence):
60     print("The sentence is a palindrome.")
61 else:
62     print("The sentence is not a palindrome.")
63
64
65
66
```

```
File "c:\Users\hp\OneDrive\Desktop\AI\lab-8.3.py", line 60
"
^
SyntaxError: unterminated string literal (detected at line 60)
PS C:\Users\hp\OneDrive\Desktop\AI> & C:/Users/hp/AppData/Local/Microsoft/WindowsApps/python3.11.exe c:/Users/hp/OneDrive/Desktop/AI/lab-8.3.py
File "c:\Users\hp\OneDrive\Desktop\AI\lab-8.3.py", line 60
    A man, a plan, a canal, Panama",
    ^
SyntaxError: unterminated string literal (detected at line 60)
PS C:\Users\hp\OneDrive\Desktop\AI> & C:/Users/hp/AppData/Local/Microsoft/WindowsApps/python3.11.exe c:/Users/hp/OneDrive/Desktop/AI/lab-8.3.py
Enter a sentence: a dog a cat a tiger
The sentence is not a palindrome.
PS C:\Users\hp\OneDrive\Desktop\AI>
```

Task 4: ShoppingCart Class

Scenario: You are designing a basic shopping cart module for an e-commerce application.

Prompt used:

generate a python function for shopping cart. the class must include add_item(name,price)

remove_item(name), total_cost(). validate correct addition, removal and cost calculation.

also handle empty cart scenarios. take user input

```
lab-8.3.py
lab-8.3.py > ...
49 # generate a python function for shopping cart. the class must include add_item(name,price)
50 # remove_item(name), total_cost(). validate correct addition, removal and cost calculation.
51 # also handle empty cart scenarios. take user input
52 #for adding give numbers like 1
53 class ShoppingCart:
54     def __init__(self):
55         self.cart = {}
56
57     def add_item(self, name, price):
58         if name in self.cart:
59             self.cart[name] += price
60         else:
61             self.cart[name] = price
62
63     def remove_item(self, name):
64         if name in self.cart:
65             del self.cart[name]
66         else:
67             print("Item not found in cart.")
68
69     def total_cost(self):
70         return sum(self.cart.values())
71
72 # Example usage
73 cart = ShoppingCart()
74 while True:
75     action = input("Enter action (add/remove/total/exit): ").lower()
76     if action == 'add':
77         name = input("Enter item name: ")
78         price = float(input("Enter item price: "))
79         cart.add_item(name, price)
80     elif action == 'remove':
81         name = input("Enter item name to remove: ")
82         cart.remove_item(name)
83     elif action == 'total':
84         print(f"Total cost: {cart.total_cost()}")
85     elif action == 'exit':
86         break
87     else:
88         print("Invalid action. Please try again.")
89
90
```

```
The sentence is not a palindrome.
PS C:\Users\hp\OneDrive\Desktop\AI> & C:/Users/hp/AppData/Local/Microsoft/WindowsApps/python3.11.exe c:/Users/hp/OneDrive/Desktop/AI/lab-8.3.py
Enter action (add/remove/total/exit): rice
Invalid action. Please try again.
Enter action (add/remove/total/exit): add
Enter item name: rice
Enter item price: 3500
Enter action (add/remove/total/exit): total
Total cost: 3500.0
Enter action (add/remove/total/exit): rice
Invalid action. Please try again.
Enter action (add/remove/total/exit): add
Enter item name: rice
Enter item price: 3500
Enter action (add/remove/total/exit): total
Total cost: 3500.0
Invalid action. Please try again.
Enter action (add/remove/total/exit): add
Enter item name: rice
Enter item price: 3500
Enter action (add/remove/total/exit): total
Total cost: 3500.0
Enter item name: rice
Enter item price: 3500
Enter action (add/remove/total/exit): total
Total cost: 3500.0
Enter item price: 3500
Enter action (add/remove/total/exit): total
Total cost: 3500.0
Enter action (add/remove/total/exit): total
```

Task 5: Date Format Conversion

Scenario: You are creating a utility function to convert date formats for reports.

Prompt used:

Write a Python function that converts a date from "YYYY-MM-DD" format to "DD-MM-YYYY" format

```
lab-8.3.py > ...
88
89 # Write a Python function that converts a date from "YYYY-MM-DD" format to "DD-MM-YYYY" format
90 def convert_date_format(date_str):
91     try:
92         year, month, day = date_str.split('-')
93         return f"{day}-{month}-{year}"
94     except ValueError:
95         return "Invalid date format. Please use 'YYYY-MM-DD'."
96 # Example usage
97 date_input = input("Enter a date in 'YYYY-MM-DD' format: ")
98 converted_date = convert_date_format(date_input)
99 print(f"Converted date: {converted_date}")
100
101
102
103
104
105
106
```

```
Enter action (add/remove/total/exit): exit
PS C:\Users\hp\OneDrive\Desktop\AI>
Ent Focus folder in explorer (ctrl + click) t): exit
PS C:\Users\hp\OneDrive\Desktop\AI> & C:/Users/hp/AppData/Local/Microsoft/WindowsApps/python3.11.exe c:/Users/hp/OneDrive/Desktop/AI/lab-8.3.py
Enter a date in 'YYYY-MM-DD' format: 2003-03-11
Converted date: 11-03-2003
PS C:\Users\hp\OneDrive\Desktop\AI> 2005-06-12
1987
PS C:\Users\hp\OneDrive\Desktop\AI>
```

Explanation:

1. The first snippet is a function to validate email addresses based on specific criteria.
2. The second snippet is a function to calculate letter grades based on numerical scores.
3. The third snippet checks if a given sentence is a palindrome, ignoring spaces and punctuation.
4. The fourth snippet defines a ShoppingCart class that allows users to add and remove items, calculate total cost, and display the cart contents.
5. The fifth snippet is a function that converts a date from "YYYY-MM-DD" format to "DD-MM-YYYY" format.